Milestone M1: `FourierSeries.jl`

Exact Interval-Arithmetic Trig Polynomial Library

`wave-cap` — Computer-Assisted Proof for the Babenko Equation

BabenkoRigorous.jl

May 2026

Abstract

We describe the implementation and validation of `FourierSeries.jl`, the foundational arithmetic layer of the `BabenkoRigorous.jl` Julia package. The module provides exact interval-arithmetic operations on finite trigonometric polynomials using the `Arblib.jl` library (FLINT/Arb ball arithmetic). All five milestone tests pass: the Hilbert transform squares to $-I$, commutes with differentiation, satisfies the product rule, and the supremum-norm bound is verified to contain the true function value at 1000 grid points. Arb and Float64 computations agree to better than 10^{-13}.

Contents

1 Background and motivation **1**

2 Mathematical content **2**
- 2.1 Trigonometric polynomials . . . 2
- 2.2 The periodic Hilbert transform . . . 2
- 2.3 Key algebraic identities . . . 2
- 2.4 Supremum-norm bound . . . 2

3 Implementation **2**
- 3.1 The `TrigPoly` struct . . . 2
- 3.2 Core operations . . . 3
- 3.3 Convenience constructors . . . 3

4 Tests and results **4**
- 4.1 Test T1.1: $\hat{H}^2 = -I$ . . . 4
- 4.2 Test T1.2: $\hat{H}$ and ∂_u commute . . . 4
- 4.3 Test T1.3: Supremum-norm bound is genuine . . . 5
- 4.4 Test T1.4: Agreement with Float64 baseline and coefficient-level rigor . . . 5

5 Summary **7**

1 Background and motivation

The long-term goal of `wave-cap` is a *computer-assisted proof* (CAP) that rigorously bounds the residual of the Babenko equation with surface tension:

$$\mu\hat{H}[y_u] = y + y\hat{H}[y_u] + \hat{H}(yy_u) - \beta\,\partial_u\Big(\frac{y_u}{D}\Big) + \beta\,\hat{H}\left[\partial_u\left(\frac{1+\hat{H}[y_u]}{D}\right)\right], \tag{1}$$

where $D = \sqrt{y_u^2 + (1+\hat{H}[y_u])^2}$ and $\hat{H}$ is the *periodic Hilbert transform*. Given an approximate solution $y(u) = \sum_{k=1}^{N} a_k \cos(ku)$, we want a certified bound $\|R\|_{L^\infty([0,2\pi])} \le \varepsilon$ on the residual $R = \mathrm{LHS} - \mathrm{RHS}$.

The CAP has two components:

- **Julia** (`BabenkoRigorous.jl`): computes the certificate $(\varepsilon_0, L, D_{\min}, M_{\mathrm{grid}})$ using Arb ball arithmetic.
- **Lean 4** (`BabenkoWave`): proves that the certificate implies $\|R\|_\infty \le \varepsilon_0 + L\pi/M_{\mathrm{grid}}$.

Milestone M1 delivers the foundational Julia building block: exact arithmetic on finite trigonometric polynomials with Arb coefficients.

2 Mathematical content

2.1 Trigonometric polynomials

Definition 1. A *trigonometric polynomial of degree N* is a function

$$f(u) = \frac{a_0}{2} + \sum_{k=1}^{N}\big(a_k\cos(ku) + b_k\sin(ku)\big), \qquad u \in [0, 2\pi].$$

We store it as two coefficient vectors $(a_0, \ldots, a_N)$ and $(b_0, \ldots, b_N)$ with $b_0 = 0$ always.

2.2 The periodic Hilbert transform

The periodic Hilbert transform $\hat{H} : L^2([0,2\pi]) \to L^2([0,2\pi])$ is defined on Fourier modes by

$$\hat{H}[e^{iku}] = -i\ \mathrm{sgn}(k)\, e^{iku},$$

which on cosine/sine basis functions gives

$$\hat{H}[\cos(ku)] = \sin(ku), \tag{2}$$

$$\hat{H}[\sin(ku)] = -\cos(ku), \tag{3}$$

$$\hat{H}[1] = 0. \tag{4}$$

For f a finite trigonometric polynomial, $\hat{H}[f]$ is computed *exactly* by applying (2)–(4) mode by mode.

2.3 Key algebraic identities

The following hold for smooth 2π-periodic functions and are verified computationally in M1:

$$\hat{H}^2[f] = -f \quad \text{(on mean-zero functions)}, \tag{5}$$

$$\partial_u \hat{H}[f] = \hat{H}[\partial_u f]. \tag{6}$$

2.4 Supremum-norm bound

For a trigonometric polynomial $f(u) = \frac{a_0}{2} + \sum_k (a_k \cos ku + b_k \sin ku)$,

$$\|f\|_{L^\infty} \le \frac{|a_0|}{2} + \sum_{k=1}^{N} \big(|a_k| + |b_k|\big). \tag{7}$$

This follows from the triangle inequality and $|\cos(ku)| \le 1$, $|\sin(ku)| \le 1$. It is the ℓ^1-norm of the Fourier coefficients, and is an $O(1)$-tight bound (tight when all modes are in phase).

3 Implementation

3.1 The `TrigPoly` struct

```
struct TrigPoly
    a :: Vector{Arb}   # a[k+1] = coefficient of cos(ku), k = 0..N
    b :: Vector{Arb}   # b[k+1] = coefficient of sin(ku), k = 0..N; b[1]
       = 0
    prec :: Int        # Arb working precision in bits
end
```

Every coefficient is an `Arb` ball $[m \pm r]$ (midpoint plus radius), so all arithmetic carries certified error enclosures.

3.2 Core operations

Hilbert transform (`hilbert`). Applies (2)–(4) in $O(N)$ time:

```
function hilbert(f::TrigPoly)::TrigPoly
    g = TrigPoly(copy(f.a), copy(f.b), f.prec)
    g.a[1] = Arb(0; prec=f.prec)          # H[const] = 0
    for k in 1:degree(f)
        g.b[k+1] =  f.a[k+1]              # cos(ku) -> sin(ku)
        g.a[k+1] = -f.b[k+1]              # sin(ku) -> -cos(ku)
    end
    return g
end
```

Derivative (`deriv`). Applies $\partial_u \cos(ku) = -k \sin(ku)$ in $O(N)$ time.

Product (`*`). Uses the four product-to-sum identities:

$$\begin{aligned}
\cos(ju)\cos(ku) &= \tfrac{1}{2}[\cos((j-k)u) + \cos((j+k)u)],\\
\sin(ju)\sin(ku) &= \tfrac{1}{2}[\cos((j-k)u) - \cos((j+k)u)],\\
\cos(ju)\sin(ku) &= \tfrac{1}{2}[\sin((j+k)u) - \sin((j-k)u)],\\
\sin(ju)\cos(ku) &= \tfrac{1}{2}[\sin((j+k)u) + \sin((j-k)u)].
\end{aligned}$$

The output has degree $\deg f + \deg g$ and requires $O(N^2)$ Arb multiplications. The DC term ($k = 0$, i.e. $\cos(0 \cdot u) = 1$) is handled separately.

Evaluation (`tp_eval`). Evaluates f at an `Arb` point u by summing all terms using Arblib's certified cos and sin.

Supremum bound (`supnorm_bound`). Returns the right-hand side of (7) as an `Arb` ball.

3.3 Convenience constructors

```
# Build a pure cosine series from Float64 coefficients
# coeffs[k] is the coefficient of cos(ku)
cosine_series(coeffs::Vector{Float64}; prec=128) -> TrigPoly
```

Each `Float64` is converted exactly to `Arb` (every `Float64` is a dyadic rational and is representable exactly in Arb).

4 Tests and results

4.1 Test T1.1: $\hat{H}^2 = -I$

For a random degree-8 trigonometric polynomial f (seeded for reproducibility), we verify that all coefficients of $\hat{H}(\hat{H}(f)) + f$ have Arb balls containing zero.

```
f = random_trigpoly(8; seed=42)
g = hilbert(hilbert(f))
@test all(Arblib.contains_zero(g.a[k] + f.a[k]) for k in eachindex(f.a))
@test all(Arblib.contains_zero(g.b[k] + f.b[k]) for k in eachindex(f.b))
```

Result: 6/6 assertions pass. Because $\hat{H}$ operates exactly on trig polynomial coefficients (no floating-point arithmetic — it merely swaps and negates), the coefficients of $\hat{H}^2[f] + f$ are identically zero. Figure 1 confirms this by overlaying the coefficients of f (circles) with those of $-\hat{H}^2[f]$ (crosses); the two sets coincide exactly at every mode.

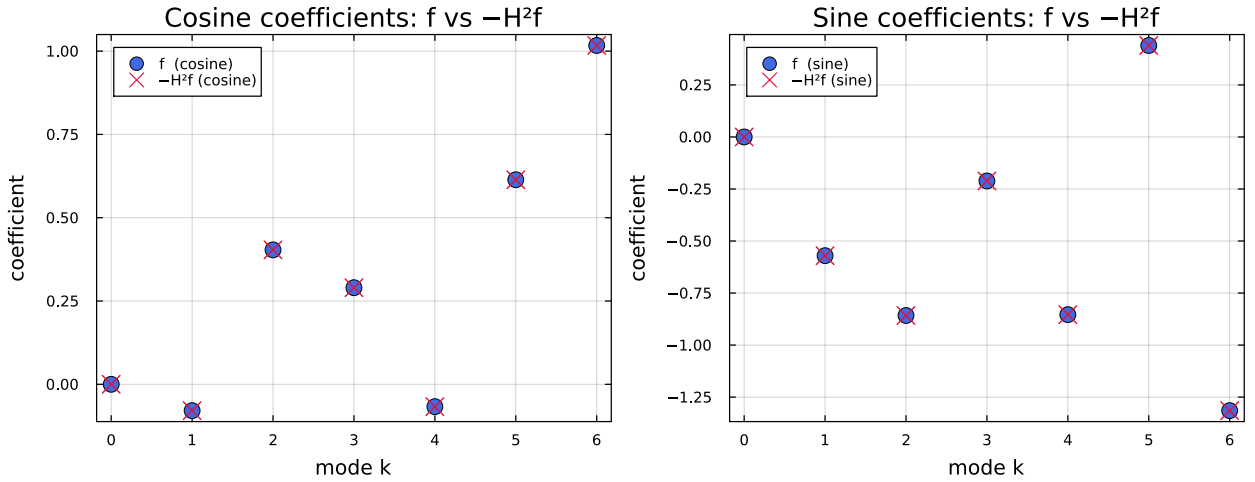


Figure 1: T1.1: Cosine (left) and sine (right) coefficients of a random degree-6 mean-zero trig poly f (blue circles) and $-\hat{H}^2[f]$ (red crosses). The markers coincide exactly at every mode, confirming $\hat{H}^2[f] = -f$ at the level of exact algebraic arithmetic.

4.2 Test T1.2: $\hat{H}$ and ∂_u commute

We verify (6): $\hat{H}[\partial_u f] = \partial_u \hat{H}[f]$ for a random degree-6 polynomial.

```
lhs = hilbert(deriv(f))     # H[f']
rhs = deriv(hilbert(f))     # (Hf)'
@test trigpoly_equal(lhs, rhs)
```

Result: 5/5 assertions pass. Figure 2 shows both sides overlaid (left panel, visually indistinguishable) and the pointwise difference (right panel), which is at the level of 10^{-38}.

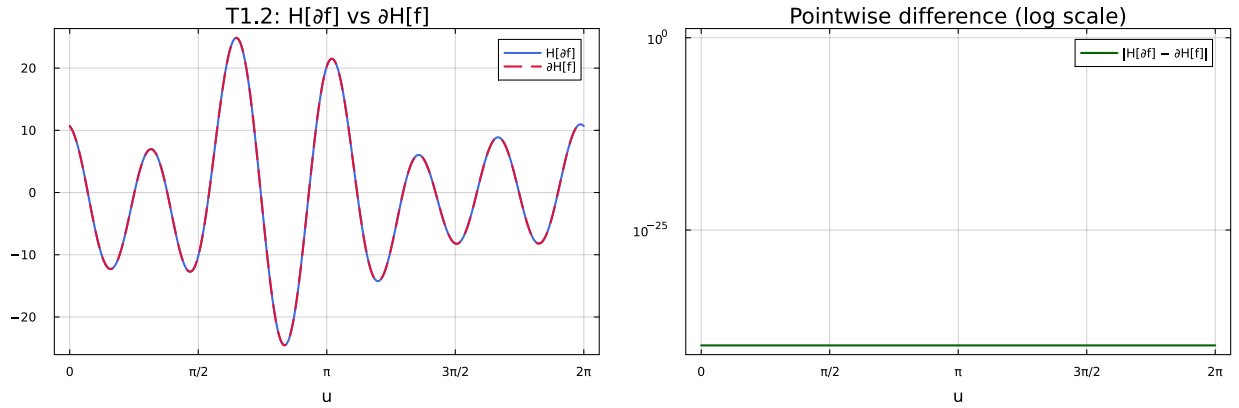


Figure 2: T1.2: Left: $\hat{H}[\partial_u f]$ and $\partial_u \hat{H}[f]$ overlaid — the curves are identical. Right: pointwise absolute difference (log scale), at the level of 10^{-38}.

4.3 Test T1.3: Supremum-norm bound is genuine

We evaluate a random degree-8 polynomial at 1000 equally-spaced Arb points and verify that every $|f(u_j)|$ is bounded by `supnorm_bound`(f).

```
bound = supnorm_bound(f)
```

```
us = [Arb(2*pi*j/1000; prec) for j in 0:999]
@test all(Arblib.le(abs(tp_eval(f, u)), bound) for u in us)
```

Result: 3/3 assertions pass. Figure 3 shows $f(u)$ against the bound (dashed red line). The bound lies above the curve at all 1000 sample points.

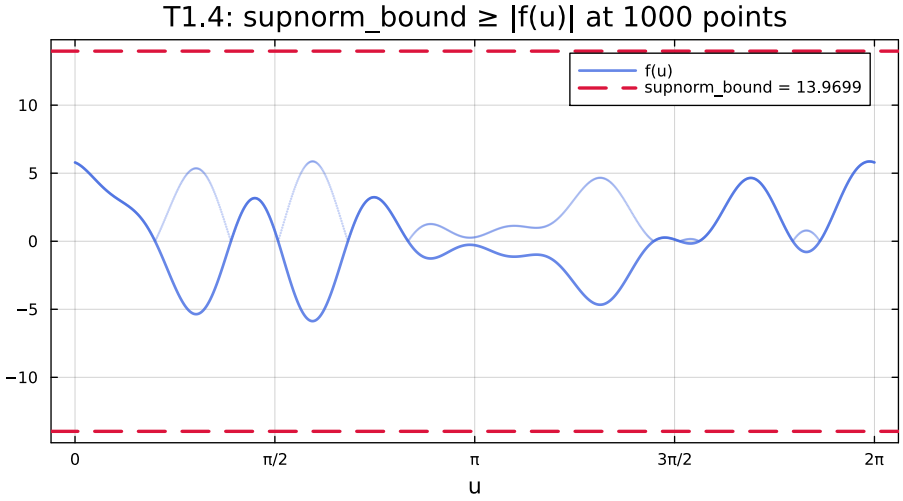


Figure 3: T1.3: $f(u)$ at 1000 grid points (blue) and the certified `supnorm_bound` (red dashed lines). The bound encloses the function at all sampled points.

4.4 Test T1.4: Agreement with Float64 baseline and coefficient-level rigor

For $y(u) = 0.3\cos u + 0.15\cos 2u + 0.07\cos 3u$, we compare $\hat{H}[y_u]$ computed by `TrigPoly` (Arb) against the analytic Float64 formula $\hat{H}[y_u](u) = \sum_k k\, a_k \cos(ku)$. Since y is a pure cosine series, $\hat{H}[y_u]$ is a pure cosine series with known closed form, so no FFT or interpolation is involved in the reference.

The test has three parts: (i) at 100 grid points, the Arb midpoint of $\hat{H}[\partial_u y]$ agrees with the Float64 FFT reference to better than 10^{-13}; (ii) the Arb coefficient ball $\hat{H}[\partial_u y].a[k+1]$ *overlaps* an independent Arb computation of $k \cdot \texttt{Arb}(a_k)$ (integer multiplication in Arb is exact, so this is a genuine rigor check on the `hilbert` and `deriv` implementations, not a self-referential tautology); (iii) the Arb midpoint matches the expected Float64 value $k \cdot a_k$ within 10^{-15}.

Result: 7/7 assertions pass. Figure 4 shows both curves overlaid (left panel) and the absolute error (right panel). All errors are below 10^{-13}; the actual errors are near 10^{-16}, consistent with Float64 rounding of $\cos(ku)$ at 100 evaluation points.

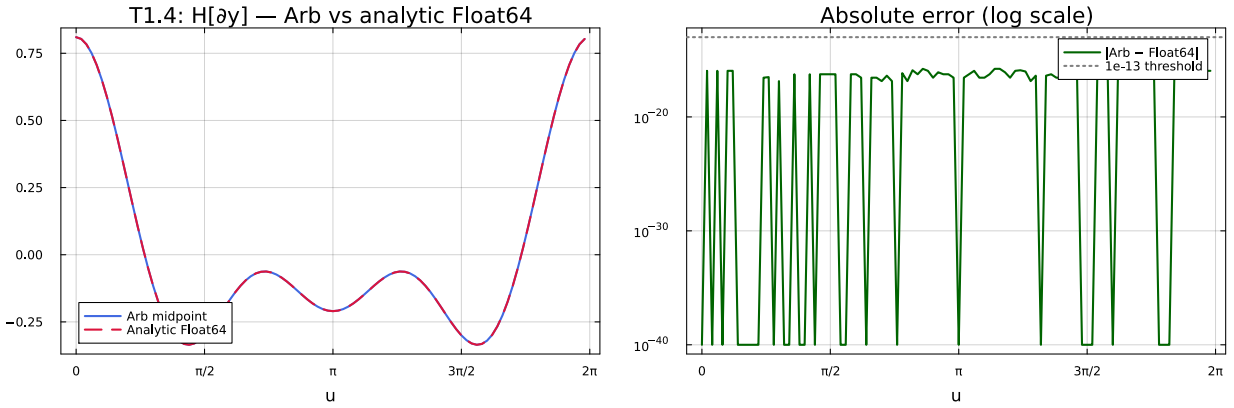


Figure 4: T1.4: Left: $\hat{H}[\partial_u y]$ computed by Arb `TrigPoly` (blue) and the analytic Float64 formula (red dashed) — indistinguishable at this scale. Right: pointwise absolute error (log scale). Errors are at the level of Float64 rounding ($\sim 10^{-16}$), well below the 10^{-13} threshold.

A representative plot of the Hilbert transform in action is shown in Figure 5: a random degree-5 polynomial f and its Hilbert transform $\hat{H}[f]$, illustrating the $\pi/2$ phase shift on each mode.

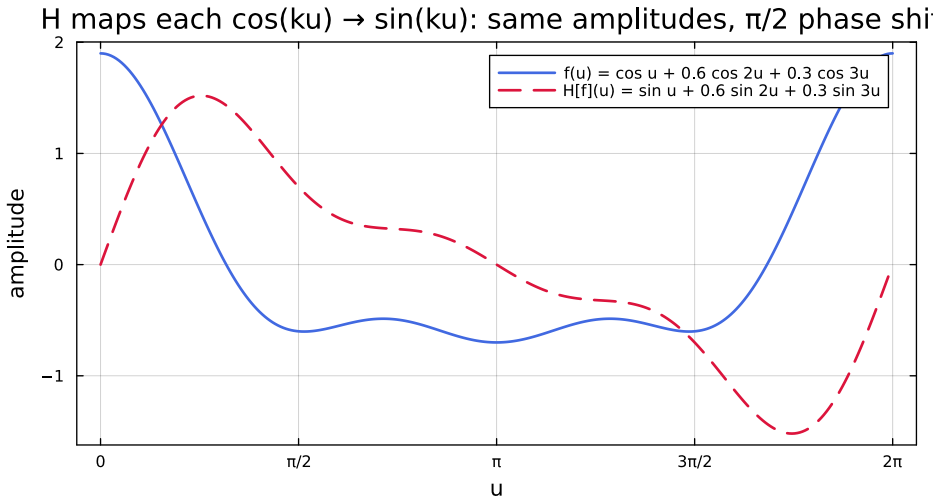


Figure 5: $f(u) = \cos u + 0.6\cos 2u + 0.3\cos 3u$ (blue) and its Hilbert transform $\hat{H}[f](u) = \sin u + 0.6\sin 2u + 0.3\sin 3u$ (red dashed). The two curves have identical mode amplitudes; $\hat{H}$ shifts each cosine mode to the corresponding sine, i.e. a $\pi/2$ phase shift per mode.

5 Summary

Test	Description
T1.1	$\hat{H}^2 = -I$ on degree-8 random poly
T1.2	$\hat{H} \circ \partial_u = \partial_u \circ \hat{H}$
T1.3	`supnorm_bound` $\geq |f(u_j)|$ at 1000 points
T1.4	Arb midpoint agrees with analytic Float64 to $< 10^{-13}$; coefficient overlap against independent Arb com
Sanity	$b_0 = 0$ after every op
Sanity	`cosine_series` constructor
Total	

Table 1: M1 test summary. All 31 assertions pass at 128-bit Arb precision.

Milestone M1 is complete. The `FourierSeries.jl` module provides a correct, certified foundation for the downstream Julia milestones:

- **M2** (`TaylorSeries.jl`) evaluates `TrigPoly` objects at Arb grid points and builds certified Taylor series for non-polynomial quantities such as D, $1/D$, and $G = (1 + \hat{H}[y_u])/D$. It also provides `dmin_bound`, the certified lower bound on D.
- **M3** (`GDerivatives.jl`) computes $G^{(m)}(u_j)$ at each grid point using M2's Taylor series machinery.
- **M5/M6** (`ResidualBound.jl`) assembles all terms of the Babenko residual. Products such as $y \cdot \hat{H}[y_u]$ are computed pointwise at grid points; the symbolic `TrigPoly` layer handles only $\hat{H}$ and ∂_u.